

Standard library hardening

Document #: P3471R0
Date: 2024-10-15
Project: Programming Language C++
Audience: Library Evolution, SG23
Reply-to: Konstantin Varlamov
<varconst@apple.com>
Louis Dionne
<ldionne@apple.com>

§ 1 Introduction

This paper proposes introducing *standard library hardening* into the C++ Standard. Hardening allows turning some instances of undefined behavior in the standard library into a guaranteed termination of the program. This proposal is based on our experience implementing [hardening in libc++](#).

§ 2 Motivation

There has been significantly increased attention to safety and security in C++ over the last few years, as exemplified by the well-known [White House report](#) and numerous recent security-related proposals.

While it is important to explore ways to make *new* code safer, we believe that the highest priority to deliver immediate real-world value should be to make *existing* code safer with minimal or no effort on behalf of users. Indeed, the amount of existing security-critical C++ code is so large that rewriting it or modifying it is both economically unviable and dangerous given the risk of introducing new issues.

There have been a few proposals accepted recently that eliminate some cases of undefined behavior in the core language. The standard library also contains many instances of undefined behavior, some of which is a direct source of security vulnerabilities; addressing those is often trivial, can be done with low overhead and almost no work on behalf of users.

In fact, at the moment all three major library implementations have some notion of a hardened or debug mode. This clearly shows interest, both from users and from implementors, in having a safer mode for the standard library. However, we believe these efforts would be vastly more useful if they were standardized and provided portable,

cross-platform guarantees to users; as it stands, implementations differ in levels of coverage, performance guarantees and ways to enable the safer mode.

Finally, leaving security of the library to be a pure vendor extension fails to position ISO C++ as providing a credible solution for code bases with formal security requirements. We believe that formally requiring the basic safety guarantees that most implementations already provide in one way or another could make a significant difference from the point of view of anyone writing or following safety and security coding standards and guidelines.

§ 3 Design overview

At a high level, this proposal consists of two parts:

- Annotate *some* existing library preconditions as *hardened*.
- Provide a way for the user to enable a *hardened mode*. When the hardened mode is enabled, all hardened preconditions are checked at runtime and violating them is guaranteed to terminate the program in an implementation-defined way.

There are a few important aspects to the proposed design:

- We only propose to formally check (some of the) existing function preconditions, not to add any new preconditions. As a drive-by, we also make a few existing preconditions more explicit in the specification (e.g. `vector::operator[]` only has implicit preconditions via iterator validity).
- Hardening deals exclusively with preconditions that must be checked at runtime. Thus, performance overhead is an important constraint on the solution.
- Hardening is intended to avoid requiring changes to an implementation's ABI. In particular, the intent is to make it possible for implementations to allow mixing hardened and unhardened code in the same program, if they wish to do so.
- Hardening needs to be lightweight enough to be used in production environments. In our experience, a debug-only approach is not sufficient to really move the needle security-wise. Thus, the goal is to identify the minimal set of checks that would deliver the most value to users while requiring minimal overhead, with a particular focus on memory safety as a major source of security vulnerabilities. Checking *all* preconditions in the library is an explicit non-goal, and would be impossible for many semantic requirements anyway.

To reiterate the last point, an important design principle is that hardening needs to be lightweight enough for production use by a wide variety of real-world programs. In our experience in `libc++`, a small set of checks that is widely used delivers far more value than a more extensive set of checks that is only enabled by select few users. Thankfully, many of the most valuable checks, such as checking for out-of-bounds access in standard containers, also happen to be relatively cheap.

§ 4 Hardened preconditions

To specify hardening in the Standard, this proposal introduces the notion of a *hardened precondition*. A hardened precondition is a precondition that the implementation is required to check if hardening is enabled, and violating which reliably terminates the program. Thus, adding hardening to the library largely consists of turning some of the existing preconditions into hardened preconditions in the specification. For example:

(24.7.2.2.6) Element access 24.7.2.2.6 [[span.elem](#)]

```
constexpr reference operator[](size_type idx) const;
```

¹ [Hardened](#) Preconditions: `idx < size()` is `true`.

In the initial proposal, we would prefer to focus on hardened preconditions that prevent out-of-bounds memory access, i.e., compromise the memory safety of the program. These would be some of the most valuable for the user since they help prevent potential security vulnerabilities; many of them are also relatively cheap to implement. More hardened preconditions can potentially be added in the future, but the intent is for their number to be limited to keep hardening viable for production use. Specifically, the proposal is to add hardened preconditions to:

- Accessors of sequence containers, `std::span`, `std::mdspan`, `std::string`, `std::string_view` and other similar classes that might attempt to access non-existent elements (e.g. `back()` on an empty container or `operator[]` with an invalid index).
- Modifiers of sequence containers and string classes that assume the container to be non-empty (e.g. `pop_back()`).
- Accessors of `optional` and `expected` that expect the object to be non-empty.

In our experience, hardening all of these operations is trivial to implement and provides significant security value.

§ 5 Enabling hardening

Implementations are required to provide a hardened mode. However, the way this hardening mode is enabled is implementation-defined, and so are details like whether the mode can be selected on a per-TU basis or needs to be the same program-wide. For example, a valid implementation would be to provide a compiler flag like `-flibrary-hardening` or to provide a library macro. If this proposal gets accepted, we expect implementations to converge on a portable mechanism.

If a hardening check fails, the program is terminated in an implementation-defined way. We don't propose allowing this behavior to be customized by the user. Calling

`std::abort()` or `__builtin_trap()` would be a valid implementation; if Contracts are available, using contract assertions would also be a valid implementation strategy.

§ 6 Relationship to Contracts, Profiles, and Erroneous Behavior

Hardening has some overlap with the recently-adopted Erroneous Behavior, as well as two other in-progress proposals: Contracts and Profiles.

§ 6.1 Contracts

Contracts is a new language feature that allows expressing preconditions and much more, with a lot of flexibility on what happens when an assertion is not satisfied. Using contract assertions is one valid implementation strategy for hardening, but the intent is to allow for other implementation strategies as well. As such, this proposal is independent from contracts.

Note that the mechanism for enabling the hardened mode would need to be different from the one for choosing the evaluation semantic – the hardened mode affects only hardened preconditions in the library, while the evaluation semantic affects any contract assertions in the program.

§ 6.2 Profiles

The Profiles proposal introduces a framework to specify sets of safety guarantees (such as a type safety profile or an invalidation profile). If profiles become a part of the Standard in the future, hardening can most likely be formulated as an additional profile; this would formalize how hardening is turned on and off. However, specifying hardened preconditions standardizes existing practice and delivers value today without requiring a new language feature.

While the current proposal does leave a bit more than we'd like to be implementation-defined, we don't expect this to significantly hinder portability since we expect implementers to converge on how to enable hardening.

§ 6.3 Erroneous Behavior

While Erroneous Behavior is a way to clearly mark some code as incorrect in the specification, it does not provide any termination guarantees. Implementations are only encouraged, but not required, to diagnose EB. Since a major aim of this proposal is to provide portable guarantees for the library, it is not sufficient to simply turn certain library precondition violations into EB.

§ 7 Suggested polls

- Are we interested in standardizing some notion of hardening for the Standard Library?
- Are we interested in forcing a conforming implementation to provide a hardened mode (as opposed to permitting that it provides one)?